

Zach Lowe, EECS 348

Gen AI Code Comparison - Assignment 2

In this assignment, I chose to compare the way in which Copilot and DeepSeek fulfilled the following prompt while adhering to the attached 2 images (see figures 1 & 2) as guidelines for them to follow:

I need you to write a program in C that will prioritize emails for a busy company CEO. You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status. You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO. You must create functions from scratch. Do not include pre-existing heap modules.

- Test File format:

Command	Description
EMAIL <sender category>, <subject line>, <date>	The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows: • Boss – read first • Subordinate – read next • Peer – read next • ImportantPerson – read next • OtherPerson – read last If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint. EMAIL is followed by space. The rest of the fields are delimited. Assume <sender category> is one of the five strings listed above. Assume <subject line> is a string which may contain spaces, but not commas Assume <date> is in the format: MM-DD-YYYY
NEXT	Next email for the CEO to read. Display the information on the terminal in the following format: Sender: Subject: Date:
READ	CEO has read email and dealt with it
COUNT	Display current count of unread emails

- The grader will test your code with a different test file. The test file will begin with the emails in the CEO's Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

- Sample Test File:

```
EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL OtherPerson,Try our product,12-19-2024
EMAIL Boss,Important,12-20-2024
EMAIL Subordinate,How do I handle this?,12-25-2024
EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024
EMAIL Boss,Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT
```

- Your program should display this:

```
There are 6 emails to read.

Next email:
Sender: Boss
Subject: Never Mind
Date: 01-03-2025

Next email:
Sender: Boss
Subject: Important
Date: 12-20-2024

There are 4 emails to read.
```

Figures 1 & 2: Prompt Images

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  #define MAX_EMAILS 1000
6  #define MAX_SUBJECT 256
7
8  typedef struct {
9      char sender[30];
10     char subject[MAX_SUBJECT];
11
12     int month;
13     int day;
14     int year;
15 } Email;
16
17 typedef struct {
18     Email data[MAX_EMAILS];
19     int size;
20 } MaxHeap;
21
22 /* ----- Priority Functions ----- */
23
24 int senderPriority(char *sender)
25 {
26     if(strcmp(sender, "Boss") == 0)
27         return 5;
28     if(strcmp(sender, "Subordinate") == 0)
29         return 4;
30     if(strcmp(sender, "Peer") == 0)
31         return 3;
32     if(strcmp(sender, "ImportantPerson") == 0)
33         return 2;
34     return 1; /* OtherPerson */
35 }
36
37 /* return 1 if a has higher priority than b */
38 int higherPriority(Email a, Email b)
39 {
40     int pa = senderPriority(a.sender);
41     int pb = senderPriority(b.sender);
42
43     if(pa > pb)
44         return 1;
45     if(pa < pb)
46         return 0;
47     /* Same sender category -> newer date wins */
48     if(a.year != b.year)
49         return a.year > b.year;
50     if(a.month != b.month)
51         return a.month > b.month;
52     return a.day > b.day;
53 }
54
55
56
57
58
59
60
61
62 /* ----- Heap Operations ----- */
63
64 void swap(Email *a, Email *b)
65 {
66     Email temp = *a;
67     *a = *b;
68     *b = temp;
69 }
70
71 void heapifyUp(MaxHeap *heap, int index)
72 {
73     while(index > 0)
74     {
75         int parent = (index - 1) / 2;
76
77         if(higherPriority(heap->data[index], heap->data[parent]))
78         {
79             swap(&heap->data[index], &heap->data[parent]);
80             index = parent;
81         }
82         else
83         {
84             break;
85         }
86     }
87 }
88
89 void heapifyDown(MaxHeap *heap, int index)
90 {
91     while(1)
92     {
93         int left = 2 * index + 1;
94         int right = 2 * index + 2;
95         int largest = index;
96
97         if(left < heap->size &&
98            higherPriority(heap->data[left], heap->data[largest]))
99         {
100             largest = left;
101         }
102
103         if(right < heap->size &&
104            higherPriority(heap->data[right], heap->data[largest]))
105         {
106             largest = right;
107         }
108
109         if(largest != index)
110         {
111             swap(&heap->data[index], &heap->data[largest]);
112             index = largest;
113         }
114         else
115         {
116             break;
117         }
118     }
119 }
120

```

```

121 void insertEmail(MaxHeap *heap, Email email)
122 {
123     if(heap->size >= MAX_EMAILS)
124         return;
125
126     heap->data[heap->size] = email;
127     heapifyUp(heap, heap->size);
128     heap->size++;
129 }
130
131 Email removeMax(MaxHeap *heap)
132 {
133     Email empty;
134
135     strcpy(empty.sender, "");
136     strcpy(empty.subject, "");
137     empty.month = empty.day = empty.year = 0;
138
139     if(heap->size == 0)
140         return empty;
141
142     Email maxEmail = heap->data[0];
143
144     heap->size--;
145
146     if(heap->size > 0)
147     {
148         heap->data[0] = heap->data[heap->size];
149         heapifyDown(heap, 0);
150     }
151
152     return maxEmail;
153 }
154
155 Email peekMax(MaxHeap *heap)
156 {
157     Email empty;
158
159     strcpy(empty.sender, "");
160     strcpy(empty.subject, "");
161     empty.month = empty.day = empty.year = 0;
162
163     if(heap->size == 0)
164         return empty;
165
166     return heap->data[0];
167 }
168
169 /* ----- Display ----- */
170
171 void printEmail(Email e)
172 {
173     if(strlen(e.sender) == 0)
174     {
175         printf("There are no emails to read.\n");
176         return;
177     }
178 }
179
180 assignment_2_copilot.c
181 {
182     printf("Next email:\n");
183     printf("\tSender: %s\n", e.sender);
184     printf("\tSubject: %s\n", e.subject);
185     printf("\tDate: %02d-%02d-%04d\n",
186         e.month,
187         e.day,
188         e.year);
189 }
190
191 /* ----- Parsing ----- */
192 void processEmailLine(MaxHeap *heap, char *line)
193 {
194     Email e;
195
196     char *token;
197
198     line[strcspn(line, "\n")] = '\0';
199
200     token = strtok(line + 6, ",");
201     strcpy(e.sender, token);
202
203     token = strtok(NULL, ",");
204     strcpy(e.subject, token);
205
206     token = strtok(NULL, ",");
207
208     sscanf(token,
209         "%d-%d-%d",
210         &e.month,
211         &e.day,
212         &e.year);
213
214     insertEmail(heap, e);
215 }
216
217 /* ----- Main ----- */
218
219 int main()
220 {
221     MaxHeap heap;
222     heap.size = 0;
223
224     char line[512];
225
226     while(fgets(line, sizeof(line), stdin))
227     {
228         line[strcspn(line, "\n")] = '\0';
229
230         if(strncmp(line, "EMAIL ", 6) == 0)
231         {
232             processEmailLine(&heap, line);
233         }
234         else if(strcmp(line, "NEXT") == 0)
235         {
236             printEmail(peekMax(&heap));
237         }
238     }
239
240     else if(strcmp(line, "READ") == 0)
241     {
242         removeMax(&heap);
243     }
244     else if(strcmp(line, "COUNT") == 0)
245     {
246         printf("There are %d emails to read.\n",
247             heap.size);
248     }
249 }
250
251 return 0;
252 }

```

```

238     }
239
240     else if(strcmp(line, "READ") == 0)
241     {
242         removeMax(&heap);
243     }
244     else if(strcmp(line, "COUNT") == 0)
245     {
246         printf("There are %d emails to read.\n",
247             heap.size);
248     }
249 }
250
251 return 0;
252 }

```

Figures 3, 4, 5, 6, and 7: Initial Copilot Code

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <ctype.h>
5
6 #define MAX_EMAILS 100
7 #define MAX_STR 256
8
9 // --- Data Structures ---
10
11 typedef struct {
12     char senderCategory[50];
13     char subject[MAX_STR];
14     char date[20];
15     int priorityLevel; // 1-5 (5 is highest)
16     int dateValue; // YYYYMMDD format for comparison
17 } Email;
18
19 typedef struct {
20     Email items[MAX_EMAILS];
21     int size;
22 } MaxHeap;
23
24 // --- Function Prototypes ---
25 void initHeap(MaxHeap *h);
26 int getPriorityLevel(char *category);
27 int parseDate(char *dateStr);
28 void insert(MaxHeap *h, Email e);
29 Email extractMax(MaxHeap *h);
30 void heapifyUp(MaxHeap *h, int index);
31 void heapifyDown(MaxHeap *h, int index);
32 void swap(Email *e, Email *b);
33 int isHigherPriority(Email e1, Email e2);
34 void printEmail(Email e);
35
36 // --- Main Function ---
37
38 int main() {
39     MaxHeap heap;
40     initHeap(&heap);
41
42     char line[512];
43     char command[20];
44
45     // Read input until EOF
46     while (fgets(line, sizeof(line), stdin)) {
47         // Remove newline character
48         line[strcspn(line, "\n")] = 0;
49
50         // Parse the command (first token)
51         if (sscanf(line, "%s", command) != 1) continue;
52
53         if (strcmp(command, "EMAIL") == 0) {
54             Email newEmail;
55
56             // Format: EMAIL <sender category>,<subject line>,<date>
57             // We need to parse this manually because subject can contain spaces.
58             // 1. Skip "EMAIL "
59
60             char *ptr = line + 6;
61
62             // 2. Extract Sender Category (up to comma)
63             char *comma1 = strchr(ptr, ',');
64             if (comma1 != NULL) {
65                 *comma1 = '\0'; // Null terminate sender
66                 strcpy(newEmail.senderCategory, ptr);
67                 ptr = comma1 + 1; // Move past comma
68             }
69
70             // 3. Extract Subject (up to next comma)
71             char *comma2 = strchr(ptr, ',');
72             if (comma2 != NULL) {
73                 *comma2 = '\0'; // Null terminate subject
74                 strcpy(newEmail.subject, ptr);
75                 ptr = comma2 + 1; // Move past comma
76             }
77
78             // 4. Extract Date (rest of string)
79             strcpy(newEmail.date, ptr);
80
81             // Calculate sorting keys
82             newEmail.priorityLevel = getPriorityLevel(newEmail.senderCategory);
83             newEmail.dateValue = parseDate(newEmail.date);
84
85             // Insert into Priority Queue
86             insert(&heap, newEmail);
87
88         } else if (strcmp(command, "NEXT") == 0) {
89             if (heap.size > 0) {
90                 Email e = extractMax(&heap);
91                 printf("\nNext email:\n");
92                 printf("    Sender: %s\n", e.senderCategory);
93                 printf("    Subject: %s\n", e.subject);
94                 printf("    Date: %s\n", e.date);
95             } else {
96                 // Depending on strictness, might want to print nothing
97                 // But usually a message or nothing.
98             }
99
100         } else if (strcmp(command, "READ") == 0) {
101             if (heap.size > 0) {
102                 // Remove the max priority email
103                 extractMax(&heap);
104             }
105
106         } else if (strcmp(command, "COUNT") == 0) {
107             printf("There are %d emails to read.\n\n", heap.size);
108         }
109     }
110
111     return 0;
112 }
113
114 // --- Helper Functions ---
115
116 void initHeap(MaxHeap *h) {
117     h->size = 0;

```

```

116 }
117
118 // Maps sender string to integer priority
119 // 5: Boss, 4: Subordinate, 3: Peer, 3: ImportantPerson, 1: OtherPerson
120 int getPriorityLevel(char *category) {
121     if (strcmp(category, "Boss") == 0) return 5;
122     if (strcmp(category, "Subordinate") == 0) return 4;
123     if (strcmp(category, "Peer") == 0) return 3;
124     if (strcmp(category, "ImportantPerson") == 0) return 3;
125     return 1; // OtherPerson
126 }
127
128 // Converts MM-DD-YYYY to an integer YYYYMMDD for easy comparison
129 int parseDate(char *dateStr) {
130     int month, day, year;
131     // date is in MM-DD-YYYY format
132     if (sscanf(dateStr, "%d-%d-%d", &month, &day, &year) == 3) {
133         return year * 10000 + month * 100 + day;
134     }
135     return 0; // Error case
136 }
137
138 // Compares two emails. Returns 1 if e1 has higher priority than e2.
139 int isHigherPriority(Email e1, Email e2) {
140     // 1. Compare Priority Level (Sender Category)
141     if (e1.priorityLevel != e2.priorityLevel) {
142         return e1.priorityLevel > e2.priorityLevel;
143     }
144
145     // 2. If same category, compare Date (Newer is better)
146     // "Newest email (not oldest) should be read first"
147     return e1.dateValue > e2.dateValue;
148 }
149
150 void swap(Email *a, Email *b) {
151     Email temp = *a;
152     *a = *b;
153     *b = temp;
154 }
155
156 // --- Heap Implementation ---
157
158 void insert(MaxHeap *h, Email e) {
159     if (h->size >= MAX_EMAILS) {
160         printf("Heap is full\n");
161         return;
162     }
163
164     // Place at end
165     h->items[h->size] = e;
166     h->size++;
167
168     // Bubble up
169     heapifyUp(h, h->size - 1);
170 }
171
172 Email extractMax(MaxHeap *h) {
173     if (h->size <= 0) {
174         Email empty = {"", "", "", 0, 0};
175         return empty;
176     }
177
178     Email maxItem = h->items[0];
179
180     // Move last item to root
181     h->items[0] = h->items[h->size - 1];
182     h->size--;
183
184     // Sink down
185     heapifyDown(h, 0);
186
187     return maxItem;
188 }
189
190 void heapifyUp(MaxHeap *h, int index) {
191     int parent = (index - 1) / 2;
192
193     // While we are not at root and current is higher priority than parent
194     if (index > 0 && isHigherPriority(h->items[index], h->items[parent])) {
195         swap(&h->items[index], &h->items[parent]);
196         heapifyUp(h, parent);
197     }
198 }
199
200 void heapifyDown(MaxHeap *h, int index) {
201     int largest = index;
202     int left = 2 * index + 1;
203     int right = 2 * index + 2;
204
205     // Check if left child exists and is higher priority than root
206     if (left < h->size && isHigherPriority(h->items[left], h->items[largest])) {
207         largest = left;
208     }
209
210     // Check if right child exists and is higher priority than current largest
211     if (right < h->size && isHigherPriority(h->items[right], h->items[largest])) {
212         largest = right;
213     }
214
215     // If largest is not root, swap and continue heapifying
216     if (largest != index) {
217         swap(&h->items[index], &h->items[largest]);
218         heapifyDown(h, largest);
219     }
220 }

```

Figures 8, 9, 10, and 11: Initial DeepSeek Code

When analyzing the code for correctness, both the Copilot and DeepSeek models properly implemented a MaxHeap for the priority queue. However, Copilot gives ImportantPerson a priority of 2 while DeepSeek gives it a priority of 3 making Copilot's the more correct one. Copilot also keeps the email in the queue when 'NEXT' is used, whereas DeepSeek removes it, making DeepSeek's more likely to encounter an error. For execution time/time complexity, both the Copilot and DeepSeek code have a time complexity of $O(\log n)$ for inserting and removing emails. However, DeepSeek calculates the priority and date values when an email is added, whereas Copilot calculates priority during comparisons, giving DeepSeek a very slight technical advantage on this front. For space complexity, both the Copilot and DeepSeek code have a space complexity of $O(n)$ because both store the emails in an array-based heap. Also, it's worth noting that Copilot's code allows up to 1000 emails compared to DeepSeek's 100, but this does not change the overall complexity, meaning neither has a significant advantage in this category. For maintainability, I think DeepSeek performed a bit better in that its code is shorter, and that

it has comments all throughout as opposed to just the headers that Copilot made. Overall, DeepSeek performed better in 2 of the 4 categories (execution time & maintainability), Copilot performed better only in Correctness, and then they performed at an equivalent level for space complexity.

Even though DeepSeek performed better in 2 categories compared to Copilot's 1, I still decided to go with the Copilot code because I think the quality of the differences is what's important. Specifically, the fact that Copilot's code executes closer to the specified/desired functionality and doesn't risk losing the email in certain instances. Additionally, the current advantages that DeepSeek's code possess could be easily reproduced in the revision.

In the efforts of improving correctness, execution time, and maintainability with my revision, I added more consistent comments to make the code easier to follow/maintain for someone inheriting it, I changed the email structure so the program stores priority and date as values as soon as the email is added, rather than determining them every time two emails are compared. Some other minor changes made include making the input handling safer, a small adjustment to the insertion process to make the heap operation clearer and more reliable, checking that each expected part of an email is present before using it, and making it so that the sender-priority function cannot modify what it's given.